

Dire Dawa University

Computer Science Department

Hotel Booking System for Teka Nikola Hotel Project Proposal

Name	id
Bisrat worku	1406128
Natnael berhanu	1406140
Nahom joni	1406139
Iman Yusuf	1406179
Amal Jibril	14061010
Henok geto	1406784

Acknowledgment

We would like to extend our gratitude for those who contributed to the success of this project. While our part played a vital role. This paper owes much to this individual support and guidance from numerous individuals. Though it is challenging to name everyone a special recognition must be given to those whose advice and feedback have been truly indispensable throughout the process. Our heartfelt appreciation especially goes to **Mr.Eyasu** . who served as the project advisor for his support and expert guidance.

ABBREVIATIONS AND ACRONYMS

- **PHP**: Hypertext Preprocessor
- **HTML**: Hypertext markup language
- **CSS**: Cascading style sheets
- **ERD**: Entity Relationship diagram
- **UI**: User Interface
- **CRC**: Class Responsibility Collaboration

List of Tables

- **Table 1**: Time schedule
- **Table 2**: Budget plan
- **Table 3**: CRC card for Customer
- **Table 4**: CRC card for Room
- **Table 5**: CRC card for Booking
- **Table 6**: CRC card for Admin
- **Table 7**: CRC card for report
- **Table 8**: CRC card for Reception
- **Table 10**: Use case documentation for see Available rooms
- **Table 11**: Use case documentation for Bookings
- **Table 12**: Use case documentation for View Booking history
- **Table 13**: Use case documentation for Cancel Booking
- **Table 14**: Use case documentation for Manage Booking
- **Table 15**: Use case documentation for Manage Rooms
- **Table 16**: Use case documentation for Manage Users
- **Table 17**: Use case documentation for Write Reports
- **Table 18**: Use case documentation for Update Booking to checked in
- **Table 19**: Use case documentation for Record check-in date and time
- **Table 21**: Create Profile
- **Table 22**: See Available Rooms
- **Table 23**: Make Booking / Cancel Booking

- **Table 24**: View booking history
- **Table 25**: Login
- **Table 26**: Logout
- **Table 27**: Update booking status to checked-in
- **Table 28**: Record check-in time and data
- **Table 29**: Update room status to occupied
- **Table 30**: Manage Booking
- **Table 31**: Manage rooms
- **Table 32**: Manage Users
- **Table 33**: Write Reports
- **Table 34**: Sub-system Decomposition
- **Table 35**: Persistent Data Storage for User
- **Table 36**: Persistent Data Storage for Rooms
- **Table 37**: Persistent Data Storage for Room_Type
- **Table 38**: Persistent Data Storage for Facilites
- **Table 39**: Persistent Data Storage for Customer
- **Table 40**: Persistent Data Storage for Bookings
- **Table 41**: Persistent Data Storage for Check_In
- **Table 42**: Persistent Data Storage for Check_Out

List of Figures

- **Figure 1**: Use Case Diagram
- **Figure 2**: User interface of Hilton hotel
- **Figure 3**: System use case Diagram
- **Figure 3**: Sequence Diagram for User Authentication
- **Figure 4**: Sequence Diagram for View Available Rooms
- **Figure 5**: Sequence Diagram for Make Booking
- **Figure 6**: Sequence Diagram for Check-In Process
- **Figure 7**: Sequence Diagram for Cancel Booking
- **Figure 8**: Sequence Diagram for Generate Reports
- **Figure 9**: Activity Diagram for Login and Create Account
- **Figure 10**: Activity Diagram for Room Booking
- **Figure 11**: Activity Diagram for Chick-in
- **Figure 12**: Activity Diagram for Cancel Booking and Report Generation
- **Figure 13**: Activity Diagram for Manage Rooms
- **Figure 14**: Activity Diagram for Manage Users
- **Figure 14**: Class Diagram
- **Figure 15**: Hardware/Software Mapping
- **Figure 16**: State chart Diagram for Registration
- **Figure 17**: State chart Diagram for Room Booking
- **Figure 18**: State chart Diagram for Check-in
- **Figure 19**: State chart Diagram for check-out
- **Figure 19**: State chart Diagram Room status
- **Figure 20**: Collaboration Diagram for Registration
- **Figure 21**: Collaboration Diagram for Login
- **Figure 22**: Collaboration Diagram for Room booking
- **Figure 23**: Collaboration Diagram for chick-in
- **Figure 24**: ER Diagram
- **Figure 25**: Graphical User Interface Design

Table of Contents

Acknowledgement 2

List of table 2

List of figure 4

INTRODUCTION 7

1.1 Background of the project 7

1.2 Statement of the problem 7

1.3 Objective of the project 7

1.3.1 General objectives 7

1.3.2 Specific objectives 7

1.4 Scope and Limitation of the project 7

1.4.1 Scope 7

1.4.2 Limitation 8

1.5 Significance of the project 8

1.6 Beneficiaries of the project 8

1.7 Methodologies 8

1.7.1 Data gathering methodology 8

1.7.2 Development methodology 8

1.7.3 Development Tools 9

1.8 Feasibility study 9

1.8.1 Economic feasibility 9

1.8.2 Technical feasibility 9

1.8.3 Operational Feasibility 9

1.9 Project Plan 9

1.9.1 Project Time Schedule 9

1.9.2 Budget plan 10

Reference 10

2 Requirement ELICITATION 11

2.1 Overview of the current system 11

2.2 Advantage of the Existing System 11

2.3 Drawback of existing system 11

2.4 Proposed Solutions 12
2.5 Preferred solutions 12
2.6 Domain Modeling with CRC card 12
2.7 Essential Use Case Diagram 14
2.8 Essential use case documentation 15
2.9 Essential user interface prototype 19
3 SYSTEM ANALYSIS 20
3.1Overview of the new system 20
3.2 System Requirements 20
3.2.1 Functional Requirements 20
3.2.2 Non-functional Requirements 21
3.3 System modeling 22
3.3.1 System use case diagram 22
3.3.2 System use case Diagram Documentation 23
3.3.3 Sequence Diagram 27
3.3.4 Activity Diagram 30
3.3.5 Class Diagram 36
4 SYSTEM DESIGN 37
4.1 Design Goals 37
4.2 System Architecture 38
4.3 Sub-system Decomposition 40
4.4 Hardware/Software Mapping 41
4.5 State chart Diagram 42
4.6 Collaboration Diagram 45
4.7 Deployment Diagram 46
4.8 Persistent Data Storage and Management 47
4.9 Graphical User Interface Design 50
5 CONCLUSION AND RECOMMENDATIONS 51
5.1 Conclusion 51
5.2 Recommendations 51

1. Introduction

1.1 Background of the Project

In a competitive hospitality industry an efficient hotel booking system plays a pivotal role in attracting and retaining customers. The traditional methods of room booking often result in errors and a lack of transparency for guests and hotel administrators. Therefore there is a need to develop a hotel booking system that will automate reservation processes and enhance customer satisfaction and support effective management of hotel resources.

1.2 Statement of Problem

The booking system at **Teka Nikola** hotel is handled manually. This is time consuming, expensive and inefficient where by a person has to go physically to the hotel for call the hotel for booking. Because of these the need for online booking system become high.

1.3 Objectives of the Project

1.3.1 General Objective

To design an online Hotel Booking system for **Teka Nikola** Hotel.

1.3.2 Specific Objective

1. Study and analyze the current manual reservation System
2. Identify the requirements needed
3. Design an online booking system for **Teka Nikola** Hotel.
4. Deploy, test and maintain the designed system

1.4 Scope and limitation of the project

1.4.1 Scope

a) System Users
   - Customers
   - Hotel admin
   - reception

b) Functional Scope
   - Online room search and availability checking
   - Room reservation and booking management
   - Customer registration and profile management
   - Check-in and check-out record management
   - occupancy, bookings and revenue reporting

c) Operational Scope
   - System operates for a single hotel (**Teka Nikola** Hotel)
   - Stores booking data in a centralized database

- Accessible via web browser

1.4.2 Limitations

a) Technical Limitations

- No native mobile application
- Requires internet connectivity

b) Functional Limitations

- Does not support multiple hotel branches
- No payment integration

c) Operational Limitations

- Requires stable internet connection
- System availability dependent on server uptime
- Staff training required

1.5 Significance of the project

This project will help customers of **Teka Nikola** Hotel to save time and money by making the reservations online and the administrators will be able the know how many customers they have on a daily basis and the workers will be able to monitor booking with out any paper work.

1.6 Beneficiaries of the project

- **User** will be able to save time and money by booking online
- **Admin** will be able to get occupancy, booking and revenue reporting
- **Workers** will be able to monitor booking without paperwork

1.7 Methodology

1.7.1 Data gathering Methodology

- **Interviews** Interviewing staff, management to understand the current booking workflows, identify challenges and requirements
- **Surveys** By surveying selected customers we can gather difficulties in booking, understand the expectations of online booking on the customer side
- **Observation** by observing other hotels what have their own booking system we can identify the bottlenecks.
- **Document Study** Documents such as pricing lists, room catalogs, Invoices will help us identify required data fields, come up with a proper database design and understand reporting needs

1.7.2 Development Methodology

This project will adopt waterfall development methodology. The methodology is chosen because it provides a structured approach to system development making it suitable for projects with clearly defined requirement.

1.7.3 Development Tools

- **Frontend :** HTML5, CSS3, JavaScript, Bootstrap
- **Backend :** PHP
- **Database:** MySQL
- **Server :** Apache web Server
- **Toolkit :** xampp

1.8 Feasibility Study

1.8.1.Economic feasibility

The system can improve integrity, reduce operational cost save time and the increased online visibility will improve occupancy rate and revenue, Revenue , booking , occupancy reporting will make management more efficient and the development costs are low due to the use of **open-source** tools.

1.8.2 Technical feasibility

The system can be developed using widely available technologies such as HTML, CSS, JavaScript, PHP ,MySQL and Xampp the hardware required is readily accessible (Ethiotelecom) thus making it technically feasible

1.8.3 Operational feasibility

The system aligns with existing hotel workflows and it is easy to train the staff to use the system and the simplified and efficient system improves customer service making it operationally feasible.

1.9 Project Plan

1.9.1 Project time schedule

No.	Activity	Duration	Time Frame
1	Proposal preparation and approval	1 week	Week 1
2	Requirement analysis and data gathering	1 weeks	Week 2
3	System design (ERD, UI, architecture)	2 weeks	Weeks 3–4
4	System implementation (coding)	3 weeks	Weeks 5–8
5	System testing and debugging	1 weeks	Weeks 9–10
6	Deployment and documentation	1 week	Week 11

Table 1 : time schedule

1.9.2 Budget plan

No	Cost Item	Description	Estimated Cost(Birr)
1	Software tools	Development tools, database, server software (open-source)	Free
2	Hardware	server hosting	600/year
3	Development cost	System design, coding, and testing	10,000
4	Domain	Domain name	Free(for 1 year)
5	Training	User manuals and staff orientation	Free
6	Contingency	Unexpected expenses	2000
7	Total	Total expense	12,800

Table 2 : Budget plan

References

- PHP Documentation : https://www.php.net/docs.php
- MySQL Documentation : https://dev.mysql.com/doc/
- Hotel website : https://www.diamondhoteladdisababa.com/rooms-list-view/
- Waterfall model : https://www.geeksforgeeks.org/software-engineering/waterfall-model/
- Feasibility Study : https://www.geeksfo_rgeeks.org/software-engineering/types-of-feasibility-study-in-software-project-development/

Chapter Two : REQUIREMENT ELICITATION

2.1 Overview of the Existing System

The booking process in **Teka nikola** hotel currently operates in through phone calls, walk-in requests and uses paper forms to keep records. Room availability is checked manually **guest records** , billing details are stored in separately making it hard to retrieve accurate information.

2.2 Advantage of the Existing System

The current booking system in **Teka Nikola** hotel is a manual system where the worker/Receptionist has to do both booking and check-in / check-out using paper forms. This system has it's own advantages like.

advantages of the manual booking system

- **Suitable for elderly :** Most elderly people in Dire Dawa / Ethiopia are not online or use the internet making manual booking more suitable
- **No Need for Internet Access :** The manual system can function without internet.
- **Low Operational Cost :** The manual system requires a minimal financial investment, as it does not have any hosting and domain fees
- **Direct Customer interaction :** Face to Face / phone-based reservations allow personalized customer service.

2.3 Drawback of existing system

Manual booking processes suffer from inefficiencies hinder customer convenience and hotel's optimality.

Drawback of existing system

- **Time-Consuming Process :** Handling bookings manually is time-consuming and increases staff workload especially during peak periods such as holidays
- **Poor reporting :** with a manual system it is hard to get reports about **room occupancy**, **revenue** and **Bookings** accurately and in a very short time
- **Paper Based System :** makes **storing**, **retrieving** and **securing** booking data hard and inefficient.
- **Limited Accessibility :** Customers must physically visit the hotel or make phone calls to book rooms which is inconvenient and restricts potential booking

2.4 Proposed Solutions

To address the limitations of the existing system we can use different types of solutions with each having their own advantages some of them are

1. **Web Application :** Web-Application provide a robust platform for users to browse rooms , chick availability, book rooms online. This system can be access by any smart device with a web browser **eg : laptops, phones(Android and ios)** and **tablets.**
2. **Mobile Application :** Mobile applications are designed specifically for enhancing user experience on smartphones and tablets by being optimized for touchscreen they can provide intuitive and responsive interface.
3. **Telegram Bots:** Using telegram bots to provide information about room types , room availability and booking rooms.

2.5 Preferred solutions

The preferred solutions is a web application tailored for **Teka Nokola** hotel can address inefficiencies of the manual system by providing centralized, accessible and scalable platform for managing hotel bookings.

Reasons for choosing web application

- **Universal Accessibility :** A web application can be accessed from any device with a web browser.
- **Cost-Effectiveness** : Web applications eliminate the need for multiple platform-specific applications reducing development and maintenance costs.
- **Scalability** ; A web application can be enhanced to support additional features or increased user traffic.

2.6 Domain Modeling with CRC card

CRC which stands for **Class Responsibility Collaboration** cards are used to organize information about classes their responsibilities and their collaborations within a system.

CRC Model for Teka Nikola hotel booking system

Customer	
Responsibilities	**Collaborators**
• Register and manage profile • See available rooms • Make room bookings • View booking history • Cancel bookings	• Booking • Room

Table 3 : CRC card for Customer

Room	
Responsibilities	**Collaborators**
• Store room details • Update room status	• Booking • Admin

Table 4 : CRC card for Room

Booking	
Responsibilities	**Collaborators**
• Create and manage booking • Link customer to room • Store booking dates	• Room • Admin • Customer

Table 5 : CRC card for Booking

Admin	
Responsibilities	**Collaborators**
• Manage Rooms and pricing • View and manage booking • Write reports • Manage users	• Room • Booking • Report

Table 6 : CRC card for Admin

Report	
Responsibilities	**Collaborators**
• write booking reports • Write occupancy reports • Write revenue summaries	• Booking • Admin

Table 7 : CRC card for report

Reception	
Responsibilities	**Collaborators**
• Verify guest identity and booking details • Update booking status to checked-in • Update room status to occupied • Record check-in date and time	• Booking • Room • Customer • Admin

Table 8 : CRC card for Reception

2.7 Essential Use Case Diagram

A Use case Diagram for **Teka Nikola** hotel provides a high-level view of the system's functionality by illustrating the various ways various users can interact with it.

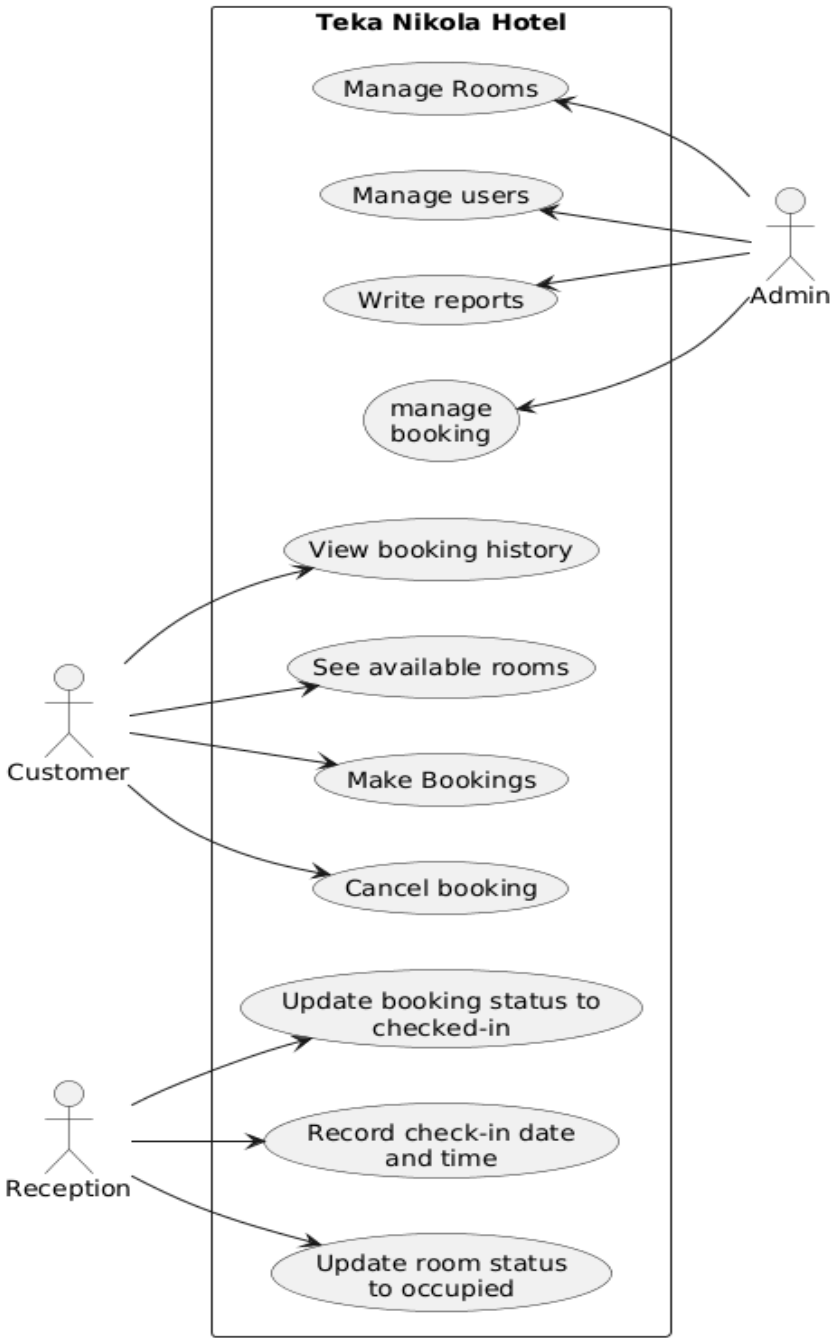


Figure 1 : Use Case Diagram

2.8 Essential use case documentation

Use Case Element	Description
Use Case Name	See Available Rooms
Use Case ID	UI01
Actor	Customer
Description	Allows guest to view available rooms
Precondition	User must connect to internet
Basic Course of Action (Flow of Events)	• Customer selects available rooms • System retrieves room data • Displays rooms
Post Condition	Available rooms are displayed
Alternative Course of Action	System displays **No rooms available**

Table 10 : Use case documentation for see Available rooms

Use Case Element	Description
Use Case Name	Bookings
Use Case ID	UI02
Actor	Customer
Description	Allows guest to book a room
Precondition	Room must be available
Basic Course of Action (Flow of Events)	• Customer selects room • Enters booking details • System creates booking
Post Condition	Available rooms are displayed
Alternative Course of Action	System tells user to choose another room

Table 11: Use case documentation for Bookings

Use Case Element	Description
Use Case Name	View Booking history
Use Case ID	UI03
Actor	Customer
Description	Displays booking history
Precondition	The customer must have booked a room before
Basic Course of Action (Flow of Events)	• Customer select booking history • System retrieves records
Post Condition	Booking history is displayed
Alternative Course of Action	System displays empty history message

Table 12: Use case documentation for View Booking History

Use Case Element	Description
Use Case Name	Cancel Booking
Use Case ID	UI04
Actor	Customer
Description	Allows guest to cancel bookings
Precondition	The booking exists
Basic Course of Action (Flow of Events)	• Customer selects booking • Customer cancels booking • System updates status
Post Condition	Booking is canceled
Alternative Course of Action	System denies cancellation

Table 13 : Use case documentation for Cancel Booking

Use Case Element	Description
Use Case Name	Manage booking
Use Case ID	UI05
Actor	Admin
Description	Allows admin to manage bookings
Precondition	Admin must be authenticated
Basic Course of Action (Flow of Events)	• Admin views booking • Admin updates booking
Post Condition	Booking updated
Alternative Course of Action	System displays error message

Table 14 : Use case documentation for Manage Booking

Use Case Element	Description
Use Case Name	Manage Rooms
Use Case ID	UI06
Actor	Admin
Description	Add, update or remove rooms
Precondition	Admin must be authenticated
Basic Course of Action (Flow of Events)	• Admin modifies room details
Post Condition	Room data updated
Alternative Course of Action	System requests correction

Table 15 : Use case documentation for Manage Rooms

Use Case Element	Description
Use Case Name	Manage Users
Use Case ID	UI07
Actor	Admin
Description	Manages system users
Precondition	Admin must be authenticated
Basic Course of Action (Flow of Events)	• Admin manages users(adds/removes)
Post Condition	User data is updated
Alternative Course of Action	System displays error message

Table 16 : Use case documentation for Manage Users

Use Case Element	Description
Use Case Name	Write Reports
Use Case ID	UI08
Actor	Admin
Description	Writes reports
Precondition	Admin must be authenticated
Basic Course of Action (Flow of Events)	• Admin selects report • System writes the report
Post Condition	Report is written
Alternative Course of Action	Writes empty report

Table 17 : Use case documentation for Write Reports

Use Case Element	Description
Use Case Name	Update Booking Status to Checked in
Use Case ID	UI09
Actor	Reception
Description	Marks bookings as checked in
Precondition	Booking must exist
Basic Course of Action (Flow of Events)	• Reception selects booking • Reception confirms arrival
Post Condition	Booking status is updated
Alternative Course of Action	System prevents duplicate action

Table 18 : Use case documentation for Update Booking to checked in

Use Case Element	Description
Use Case Name	Update room status to occupied
Use Case ID	UI10
Actor	Reception
Description	Updates room status
Precondition	Customer is checked in
Basic Course of Action (Flow of Events)	• Reception selects room • System updates status
Post Condition	Room marked occupied
Alternative Course of Action	System displays warning

Table 19 : Use case documentation for Update room status to occupied

Use Case Element	Description
Use Case Name	Record chick in date and time
Use Case ID	UI11
Actor	Reception
Description	Records guest chick in date and time
Precondition	User must connect to internet
Basic Course of Action (Flow of Events)	• Reception confirms check in • System records time
Post Condition	Check in time is saved
Alternative Course of Action	System prevents duplication

Table 19 : Use case documentation for Record chick in date and time

2.9 Essential user interface prototype

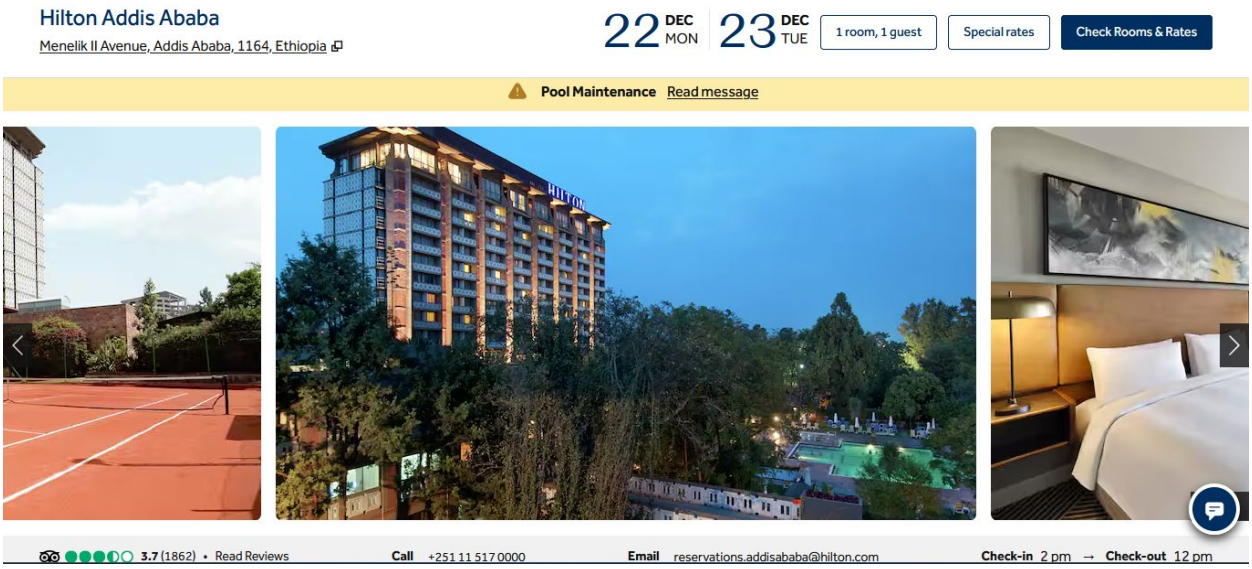


Figure 2 : User interface of Hilton hotel

Chapter Three : System Analysis

3.1 Overview of the new system

The proposed " **Booking system for Teka Nikola hotel**" is a web-based solution designed to modernize Teka Nikola hotel booking process by eliminating the inefficiencies of paper based system. while introducing Booking, revenue and occupancy reporting. The system's mobile-friendly design will ensure accessibility for users, administrators and receptionists.

This system directly addresses **Teka Nikola** hotel's challenges related to bookings. Customers gain the convenience of checking room availability , checking the facility's that come with the room and booking the room online the administrator will be able view users and get reports .

The system follows a client-server architecture, where the user interface is accessed via a web browser and the business logic is handled on server side , with all data securely stored in centralized database. Role-based access control ensures that guests and administrators can only access functions relevant to their responsibilities.

Overall the system aims to improve operational efficiency, Reduce human errors, enhance data accuracy and security, improve customer satisfaction.

3.2 System Requirements

3.2.1 Functional Requirements

- The system allows customers to sign-in and manage their profile
- The system provides admin controls for system monitoring
- The system writes reports about booking , revenue and occupation
- The system allows customers to view booking history
- The system allows customers to cancel an existing booking
- The system allows admins to log in securely
- The system allows admins to update room status
- The system allows admins to add, update and delete room information
- The system allows to record guest chick-in date and time

3.2.2 Non functional Requirements

User Interface Requirement

The system requires interaction with multiple user types having varying technical capabilities and to address the accessibility issue the interface shall maintain intuitive navigation, consistent design and responsive layouts across all devices.

Performance Requirement

The system must handle concurrent access from customers, admins and receptionists during peak periods. The system processes real-time room availability updates. Therefore the system shall maintain sub 3-second response time for all critical operations and support 100+ simultaneous users without degradation.

Security Requirements

The system should require authentication for all admin functions and shall enforce role-based access control for guests and admins. The system shall securely store user credentials using encryption and hashing techniques and protect data against unauthorized access and modification.

Availability Requirements

The system shall be available 24/7 except during scheduled maintenance and prevent data loss in case of system failure.

Maintainability Requirements

The system requires regular updates.so the system shall fallow documented coding standards, include comprehensive logging for error tracking and maintain modular component design for isolated testing and updates.

Compatibility Requirements

The system shall be compatible with modern web browsers (Chrome, Firefox, Edge) and the system shall be responsive and usable on desktop and mobile devices.

Scalability Requirements

The system shall support future expansion such as additional rooms and users also allow integration with future online payment systems.

3.3 System Modeling

3.3.1 System use case diagram

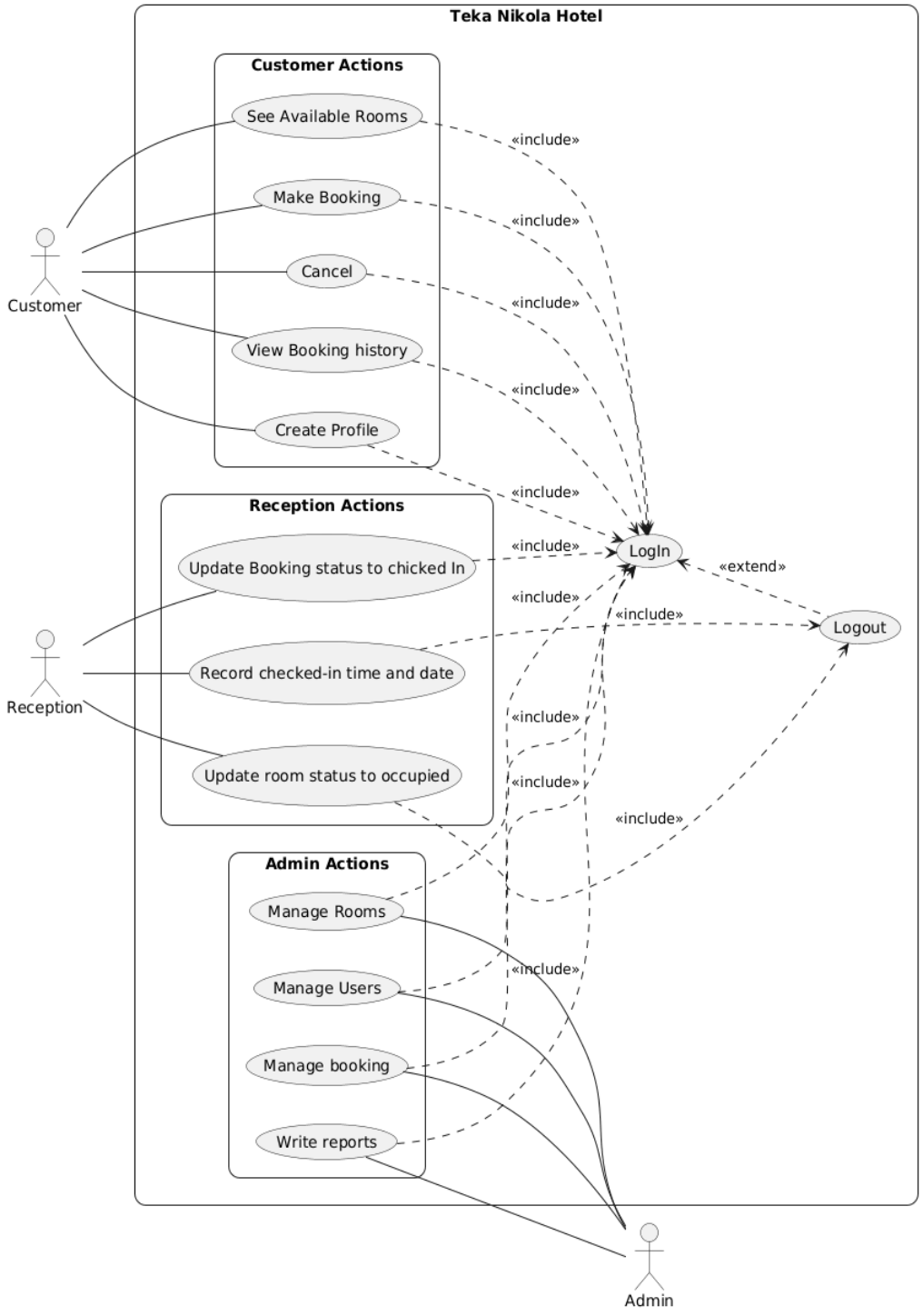


Figure 3 : System use case Diagram

3.3.2 System Use case Diagram Documentation

Use case number	UC-1
Use case Name	Create Profile
Actor	Customer
Related use case	UC-6 (login)
Description	Allows a guest to create a personal account
Precondition	Guest is not registered
Basic Flow	1. Customer opens registration page 2. Customer enters personal details 3. System validates input 4. System creates profile
Alternative course of action	If the customer enters invalid data • The system displays error

Table 21 : Create Profile

Use case number	UC-2
Use case Name	See Available Rooms
Actor	Customer
Related use case	UC-3(make booking), UC-6(login)
Description	Displays available rooms
Precondition	Customer is logged in
Basic Flow	1. Customer selects dates 2. System retrieves room data 3. System displays availability
Alternative course of action	If no rooms are available • System displays message

Table 22 : See Available Rooms

Use case number	UC-3
Use case Name	Make Booking
Actor	Customer
Related use case	UC-2(See Available Rooms), UC-6 (Login)
Description	Allows guest to book a room
Precondition	Customers logged in and room is available
Basic Flow	1. Customers selects room 2. Customers enters booking details 3. System confirms booking
Alternative course of action	If room is unavailable • Select another room

Table 23 : Make Booking

Use case number	UC-4
Use case Name	Cancel Booking
Actor	Customer
Related use case	UC-5(View Booking History), UC-6 (Login)
Description	Allows guest to cancel a booking
Precondition	Booking exists
Basic Flow	1. Customer selects booking 2. Customers confirms cancellation
Alternative course of action	If cancellation deadline passed • System denies request

Table 23 : Cancel Booking

Use case number	UC-5
Use case Name	view booking history
Actor	Customer
Related use case	UC-4 (Cancel Booking), UC-6 (Login)
Description	Displays guest booking history
Precondition	Guest logged in
Basic Flow	1. Customer selects booking history 2. System displays records
Alternative course of action	If no history found • System displays empty list

Table 24 : view booking history

Use case number	UC-6
Use case Name	Login
Actor	Customer, Receptionist, Admin
Related use case	Uc-1 (Create Profile), Uc-7 (Logout)
Description	Authenticates system users
Precondition	User has account
Basic Flow	1. User enters credentials 2. System validates credentials
Alternative course of action	If invalid credentials is entered • System displays error message

Table 25 : Login

Use case number	UC-7
Use case Name	Logout
Actor	Customers, Receptionist , Admin
Related use case	UC-6 (Login)
Description	Ends user session
Precondition	User logged in
Basic Flow	1. User selects logout 2. System ends session
Alternative course of action	None

Table 26 : Logout

Use case number	UC-8
Use case Name	Update booking status to checked-in
Actor	Receptionist
Related use case	UC-9 (Record Chick-in time), UC-10 (Update Room Status)
Description	Marks guest as checked-in
Precondition	Booking exists
Basic Flow	1. Receptionist selects booking 2. System Updates status
Alternative course of action	Booking already checked-in

Table 27 : Update booking status to checked-in

Use case number	UC-9
Use case Name	Record check-in time and data
Actor	Receptionist
Related use case	UC-8 (Update booking status), UC-10 (Update room status)
Description	Records check-in timestamp
Precondition	Booking checked-in
Basic Flow	1. System records data and time
Alternative course of action	Time is already recorded

Table 28 : Record check-in time and data

Use case number	UC-10
Use case Name	Update room status to occupied
Actor	Receptionist
Related use case	UC-8 (Update booking status), UC-9 (Record check-in time)
Description	Updates room occupancy
Precondition	Guest checked-in
Basic Flow	1. Receptionist updates room status
Alternative course of action	Room is already occupied

Table 29 : Update room status to occupied

Use case number	UC-11
Use case Name	Manage Booking
Actor	Admin
Related use case	UC-8 (Checked-in), UC-14 (Write Reports)
Description	Manages all bookings
Precondition	Admin logged in
Basic Flow	1. Admin views bookings 2. Admin updates records
Alternative course of action	Invalid booking selected

Table 30 : Manage Booking

Use case number	UC-12
Use case Name	Manage rooms
Actor	Admin
Related use case	UC-2 (See available rooms)
Description	Add, Update and delete rooms
Precondition	Admin should be logged in
Basic Flow	1. Admin modifies room data
Alternative course of action	None

Table 31 : Manage rooms

Use case number	UC-13
Use case Name	Manage Users
Actor	Admin
Related use case	UC-6 (login)
Description	Manage system users
Precondition	Admin logged in
Basic Flow	1. Admin manages users
Alternative course of action	Duplicated users

Table 32 : Manage Users

Use case number	UC-14
Use case Name	Write Reports
Actor	Admin
Relate use case	UC-11 (Manage booking), UC-12 (Manage rooms)
Description	Generates reports
Precondition	Admin logged in
Basic Flow	1. Admin selects reports 2. System generates report
Alternative course of action	The data is not available

Table 33 : Write Reports

3.3.3 Sequence Diagram

User Authentication(login)

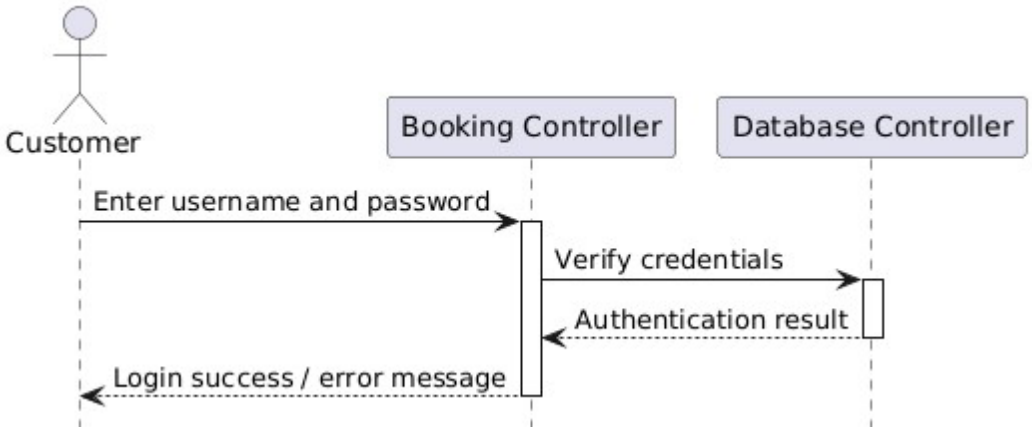


Figure 3 : Sequence Diagram for User Authentication

View Available Rooms

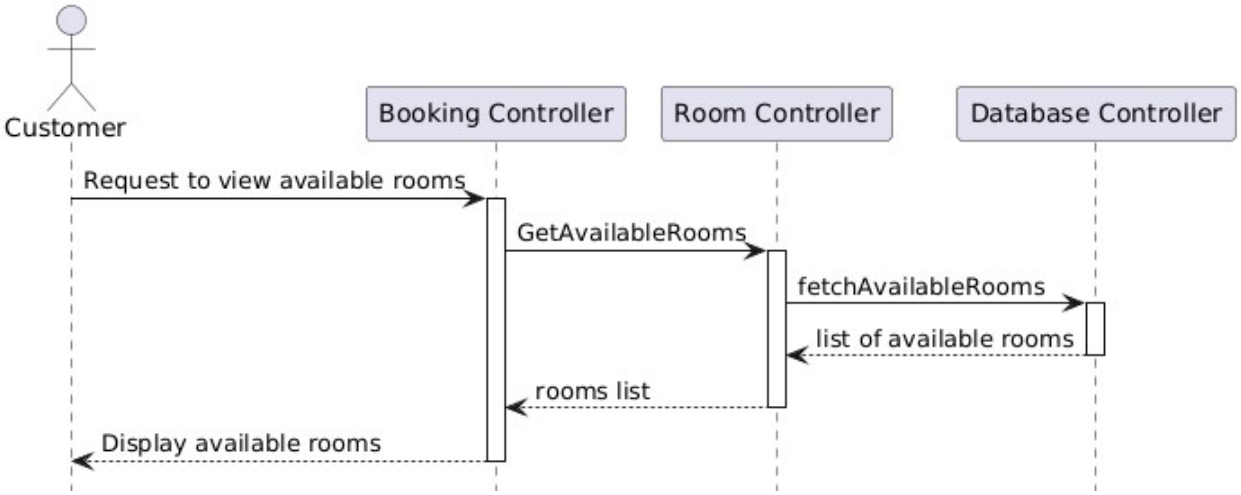


Figure 4 : Sequence Diagram for View Available Rooms

Make Booking

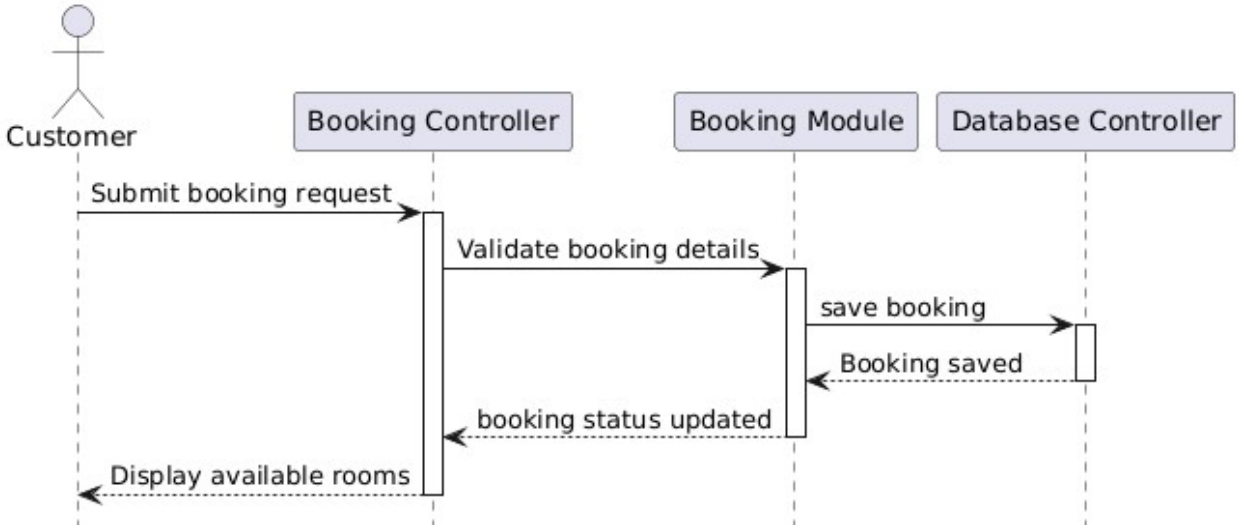


Figure 5 : Sequence Diagram for Make Booking

Check-In Process

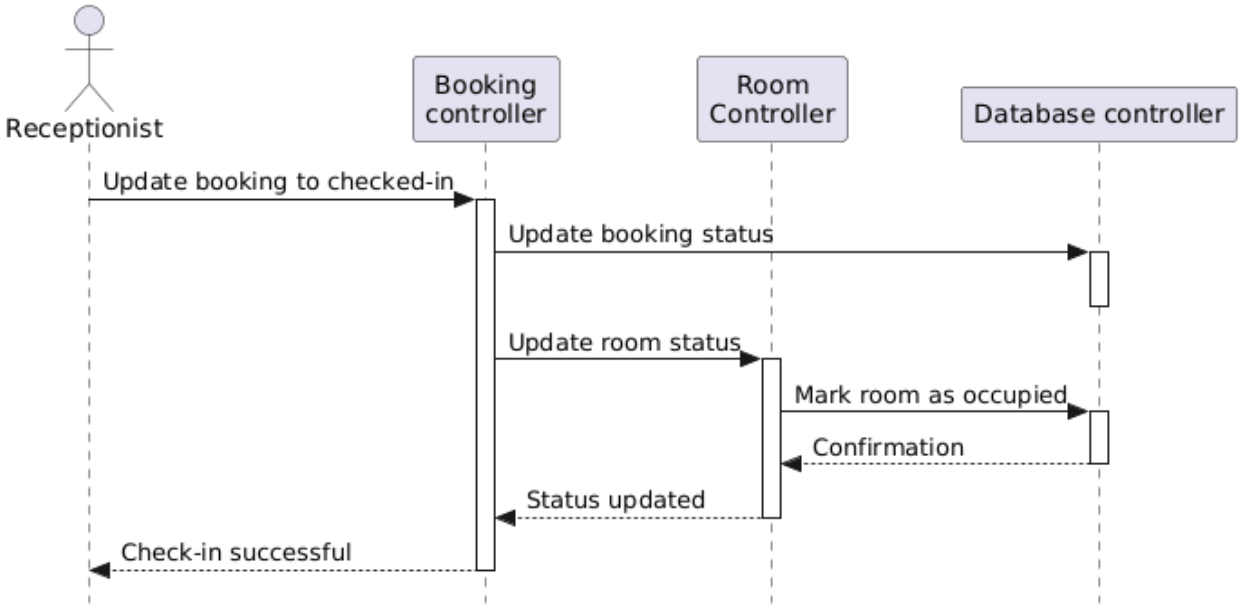


Figure 6 : Sequence Diagram for Check-In Process

Cancel Booking

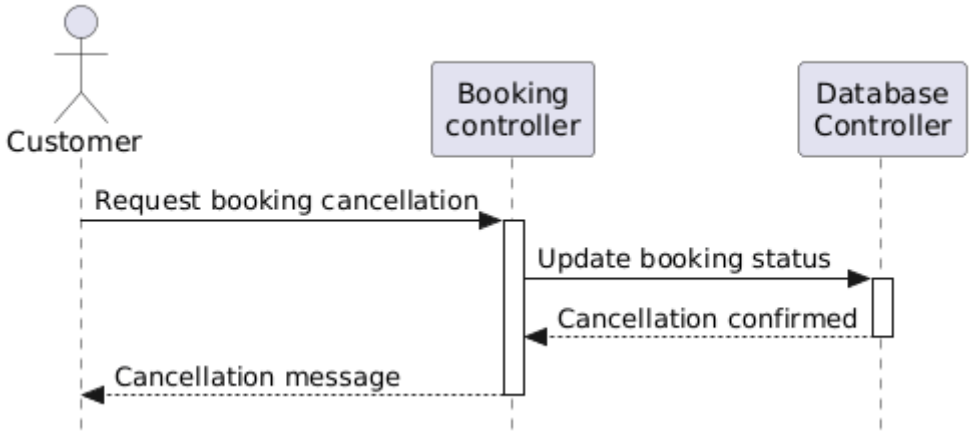


Figure 7 : Sequence Diagram for Cancel Booking

Generate Reports

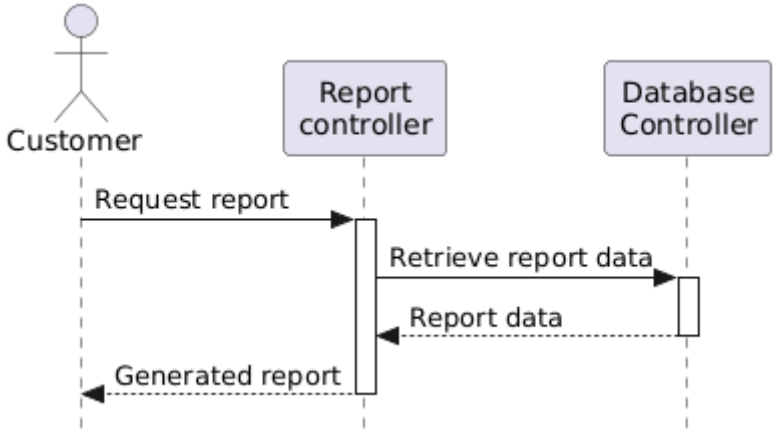


Figure 8 : Sequence Diagram for Generate Reports

3.3.4 Activity Diagram

Login and Create Account

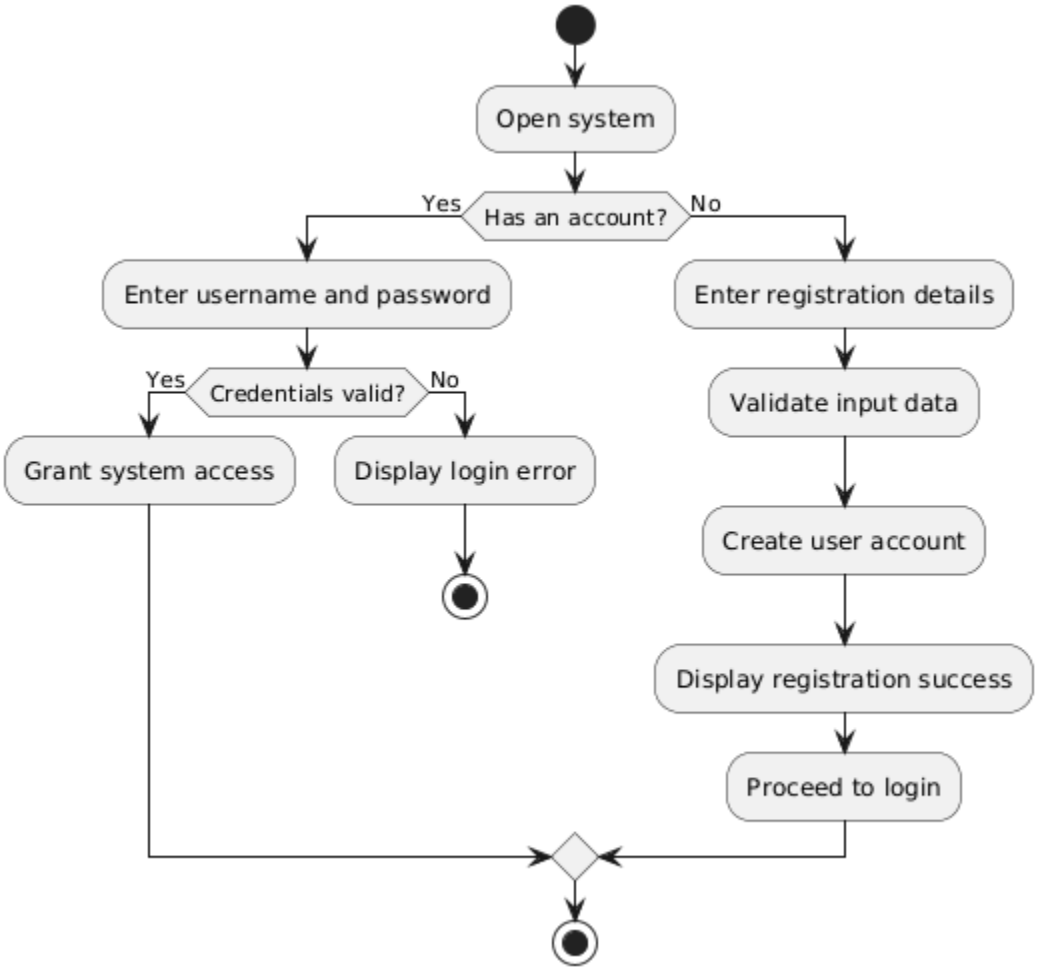


Figure 9 : Activity Diagram for Login and Create Account

Room Booking

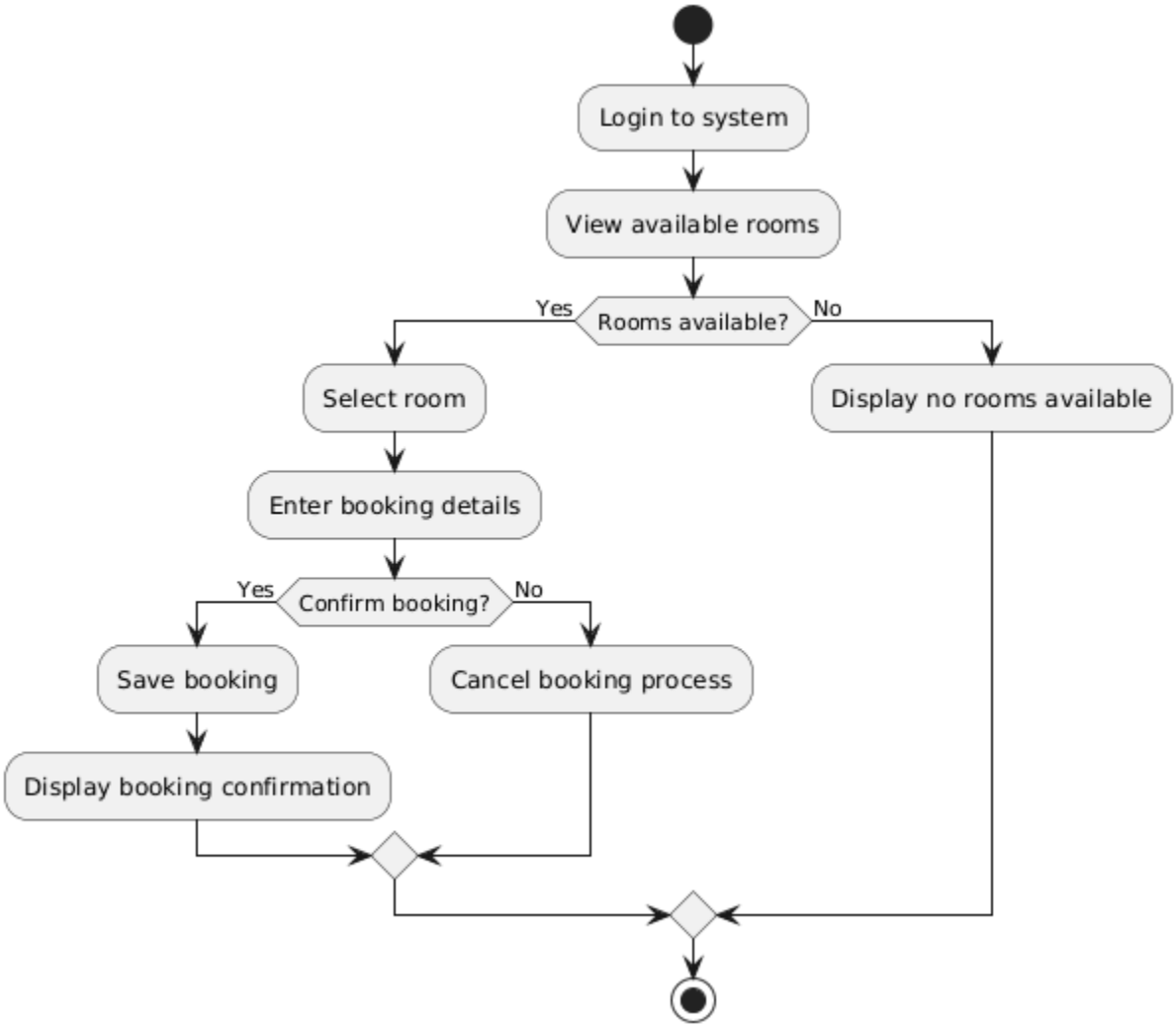


Figure 10 : Activity Diagram for Room Booking

Chick-in

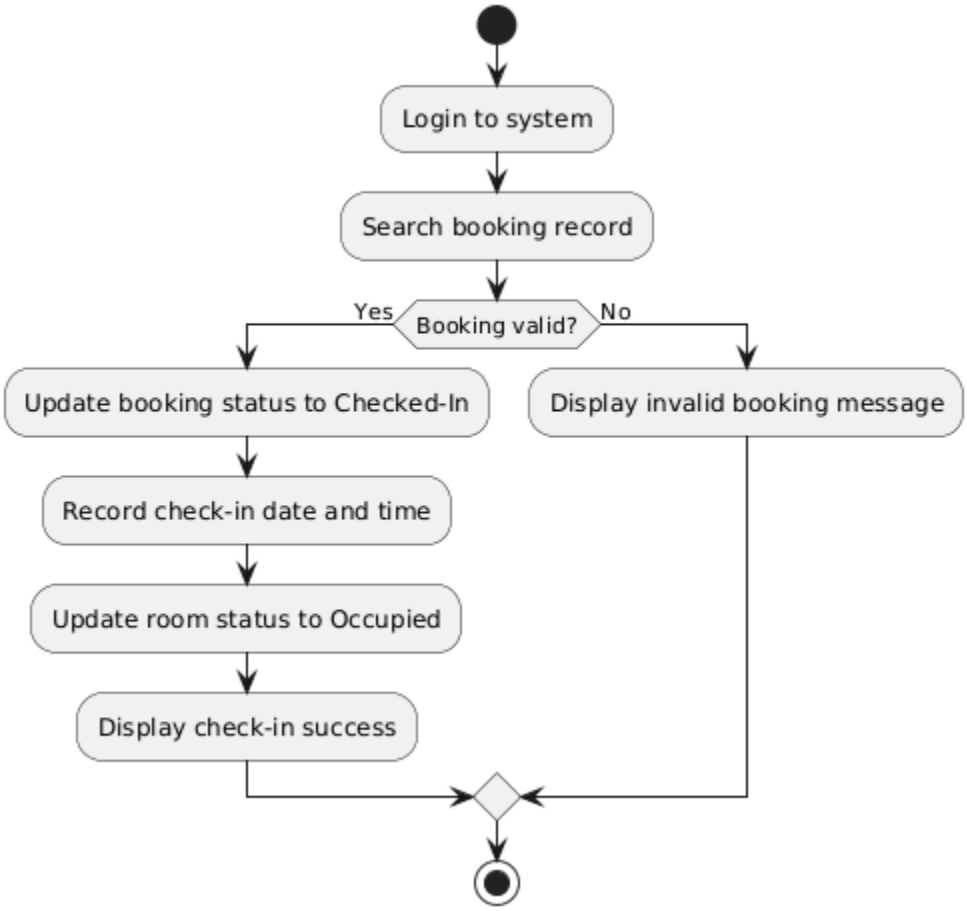


Figure 11 : Activity Diagram for Chick-in

Cancel Booking and Report Generation

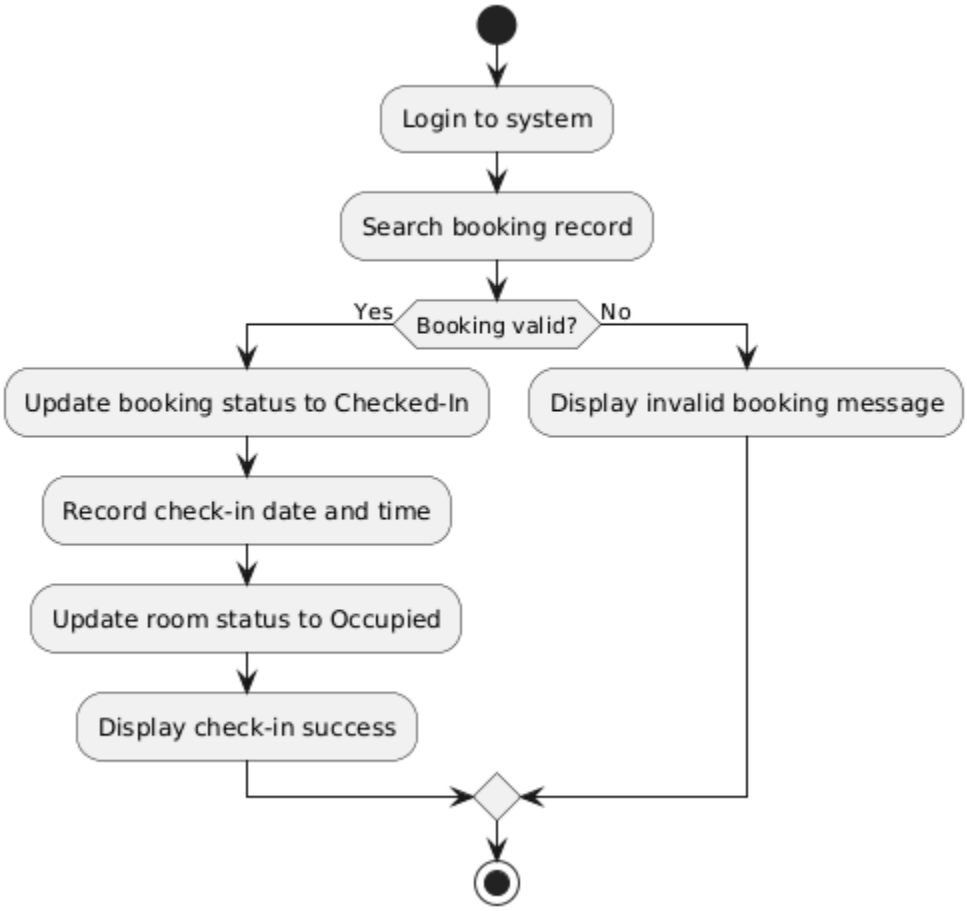


Figure 12 : Activity Diagram for Cancel Booking and Report Generation

Manage Rooms

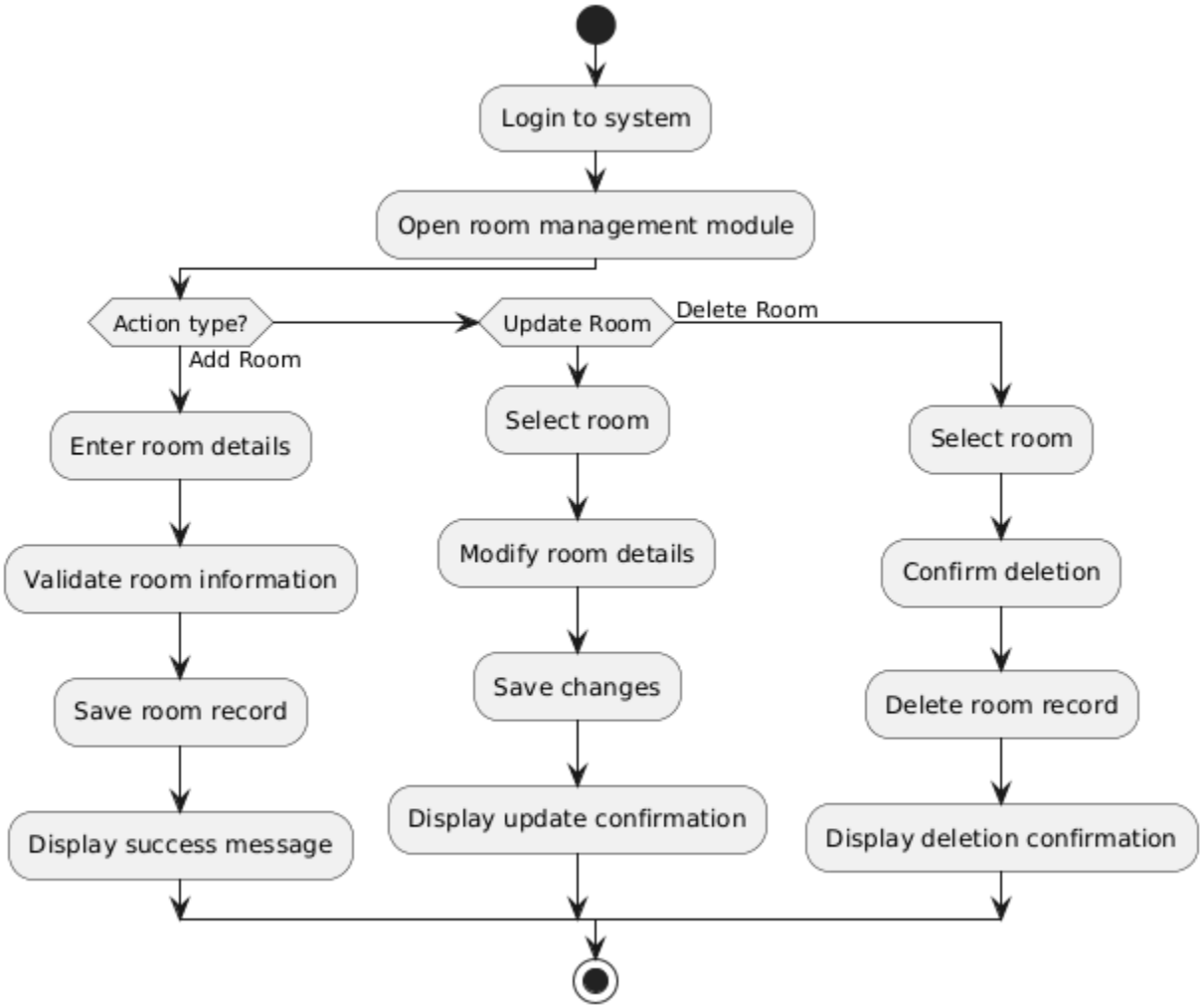


Figure 13 : Activity Diagram for Manage Rooms

Manage Users

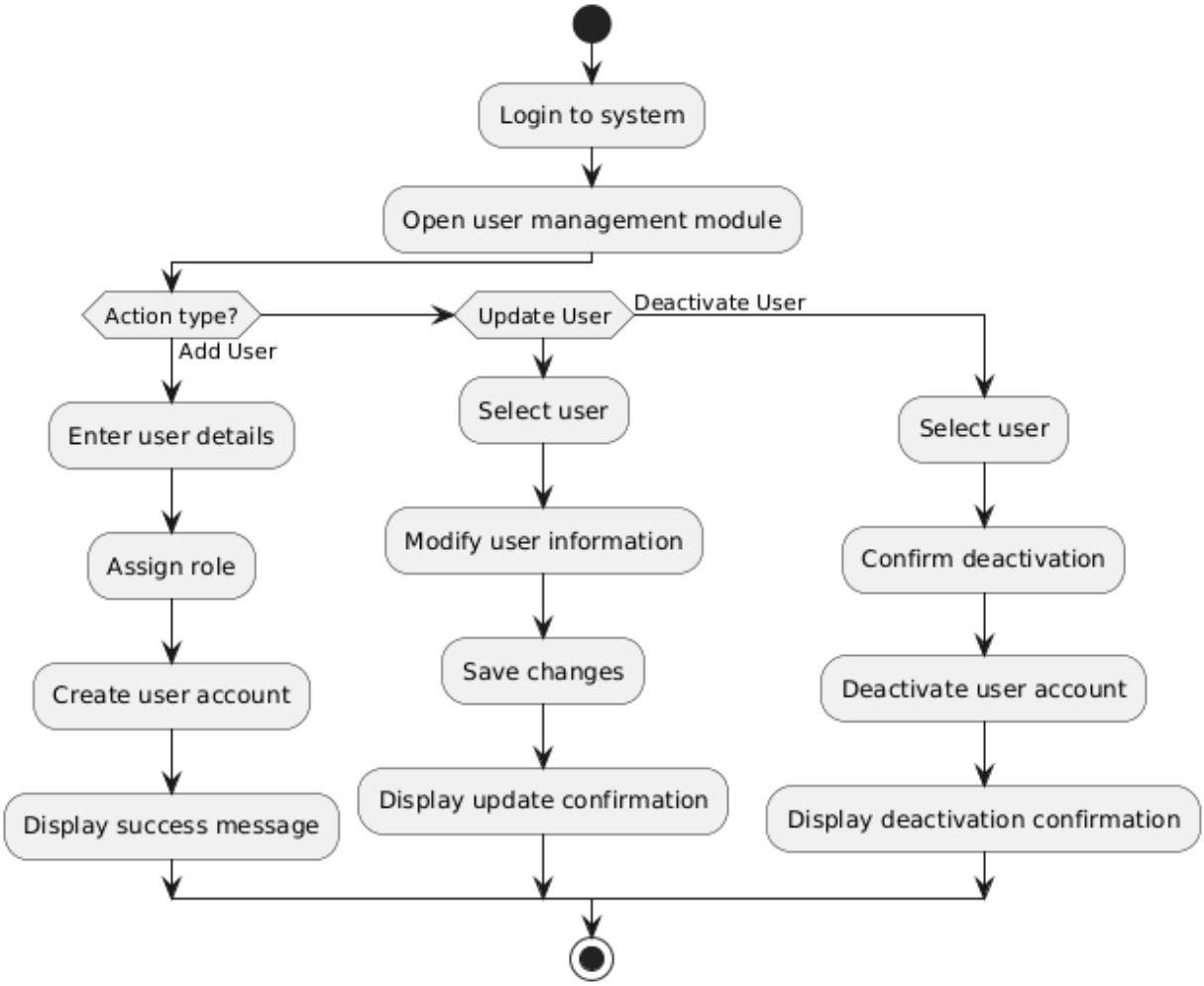


Figure 14 : Activity Diagram for Manage Users

3.3.5 Class Diagram

The Class diagram represents the core entities and relationships in Hotel booking system focusing on how different users (Customer, Admin, Reception) interact with the system components such as bookings, chick-ins, Cancellations and reports. It serves as a blueprint for understanding system structure and guiding implementation.

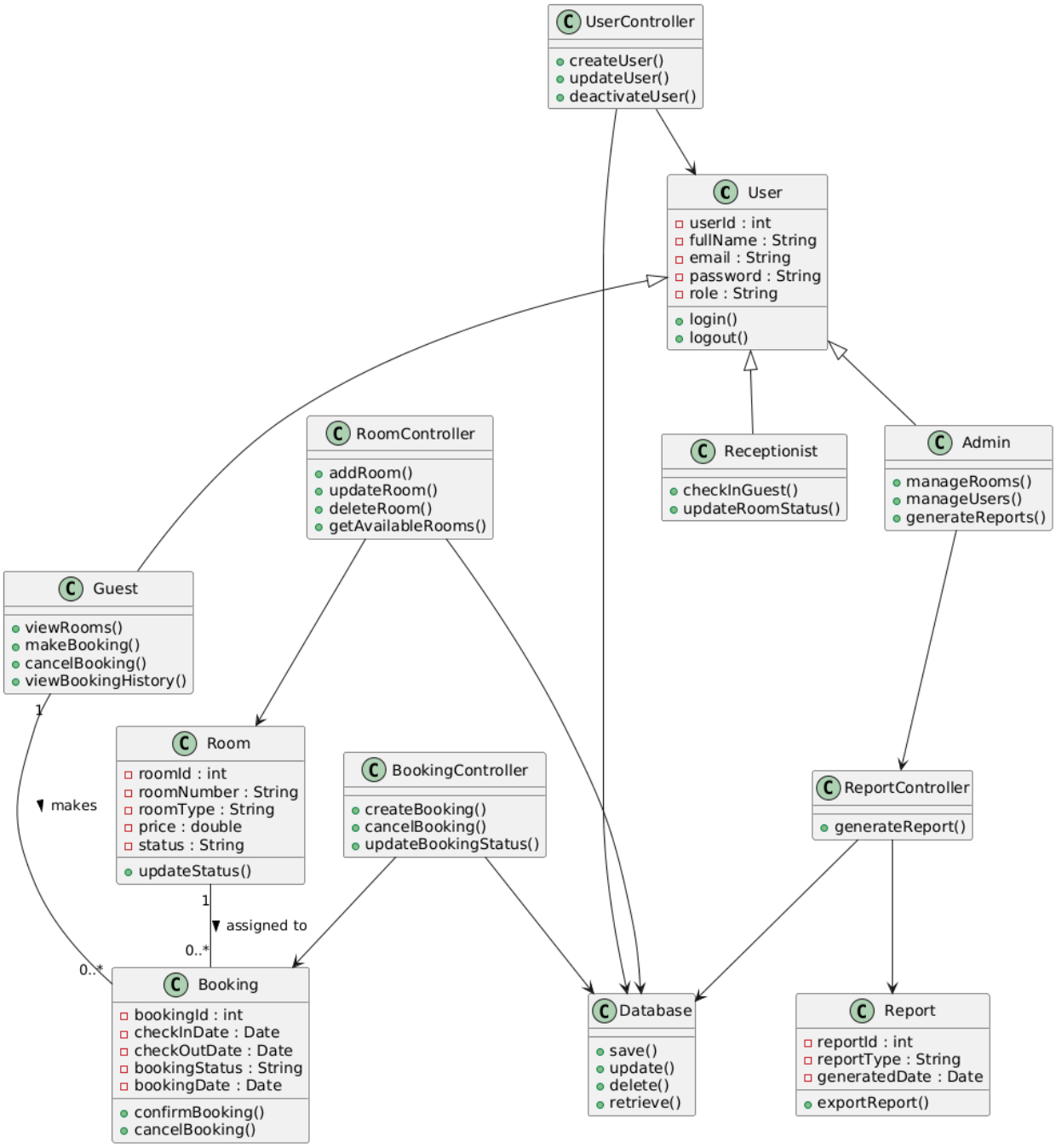


Figure 14 : Class Diagram

CHAPTER FOUR : SYSTEM DESIGN

This chapter presents the detailed design of the proposed web-based Hotel booking system for **Teka Nikola Hotel.** Building upon the system analysis conducted in the previous chapter, this section outlines the architectural blueprint, design goals and structural decomposition of the system into manageable sub-components. The goal is to transform the functional and non-functional requirements into a tangible system architecture that can guide development and implementation.

This chapter begins by defining the design goals, which capture the key qualities the system must achieve, such as usability, performance, scalability and security. Following this the system architecture is presented to illustrate the overall structure, including client-server interactions and the major software components.

4.1 Design Goals

The system design is guided by a set of high-level goals that ensure it meets the needs of users, supports growth and be reliable and secure. The design goals include:

1. Usability

The system is designed to be intuitive and easy to use for all users, including Customers , administrators, and receptionists. The user interface ensures that users can perform tasks such as room booking, check-in, and management operations with minimal training.

2. Reliability

The system ensures consistent and accurate operation by validating inputs, managing transactions properly, and preventing data inconsistency during booking and payment processes.

3. Security

The system enforces:

- Secure user authentication and authorization
- Role-based access control (Customers, Admin, Receptionist)
- Encrypted password storage
- Session management to prevent unauthorized access

4. Scalability

The system is designed to support an increasing number of users, bookings, and rooms without major architectural changes. The layered architecture allows future expansion such as online payment gateways or mobile applications.

5. Performance

Efficient database queries and optimized business logic are used to ensure fast response times for room searches, booking confirmations, and report generation.

6. Maintainability

The system follows modular and object-oriented design principles, making it easier to update, debug, and enhance individual components without affecting the entire system.

7. Availability

The system is designed to be available to users at all times, enabling guests to book rooms online and allowing staff to manage hotel operations efficiently.

8. Data Integrity

The system ensures accurate and consistent data by enforcing database constraints, validation rules, and controlled access to critical operations such as booking and payment processing.

9. Compatibility

The web-based system is compatible with modern web browsers.

10. Compliance with Requirements

The design strictly aligns with the functional and non-functional requirements defined during the system analysis phase.

4.2 System Architecture

The Hotel Booking System for "**Teka Nikola hotel"** follows a **three-tier (layered) architecture**, which separates the system into distinct layers to improve maintainability, scalability, and security. Each layer has a clearly defined responsibility and communicates with adjacent layers through well-defined interfaces.

The three main layers are:

1. Presentation Layer
2. Application (Business Logic) Layer
3. Data Layer

4.2.1 Presentation Layer

The Presentation Layer is responsible for user interaction. It provides graphical user interfaces for different system users, including **Customers, Receptionists, and Administrators**. It is accessed via web browser and It is responsible for:

- Display login and registration pages
- Allow guests to search rooms and make bookings
- Enable receptionists to manage check-ins and check-outs
- Enable administrators to manage users, rooms, and reports
- Validate basic user input before sending requests to the application layer

4.2.2 Application Layer

The Application Layer contains the core logic of the system. it processes requests from the presentation layer and applies system rules before interacting with database. Responsibilities include :

- User authentication and authorization
- Role-based access control (Admin, Receptionist, Guest)
- Booking management (create, update, cancel bookings)
- Room availability checking
- Check-in and check-out processing
- Report generation

4.2.3 Data Layer

The Data Layer is responsible for data storage and retrieval. It ensures data consistency, integrity and security. Responsibilities include :

- Store user accounts, rooms, bookings, and payment records
- Enforce data constraints and relationships
- Support data retrieval for reports and system operations

4.3 Sub-system Decomposition

Sub-system decomposition is a method used to simplify a complex system by breaking it into smaller, manageable components each responsible for a specific set of tasks. Subsystems are grouped based on their functional responsibilities such as user management, room management, process bookings and admin control.

This modular breakdown promotes **separation of concerns**, **easier maintenance** and **better scalability**. While allowing development teams to work on different components independently.

Sub-system	Description	Responsibility
User Management	Manages user accounts and roles in the system	Search rooms, create bookings, cancel bookings, view booking history
Authentication & Authorization	Controls system access and security	Verify booking, check-in guest, record time, update room status
Room Management	Manages hotel room information and availability	Add, update, delete rooms, update room status, set price
Booking Management	Handles room reservation processes	Search rooms, create bookings, cancel bookings, view booking history
Check-in / Check-out	Manages guest arrival and departure	Verify booking, check-in guest, record time, update room status
Reporting	Generates operational and management reports	Generate booking, occupancy, and user reports

Table 34 : Sub-system Decomposition

4.4 Hardware/Software Mapping

The Hotel booking system is implemented using a web-based architecture. The client-side is built with HTML, CSS, and JavaScript, accessible through modern browsers on smartphones or desktops. The server-side logic is handled by PHP, running on an Apache webserver, while the backend uses a MySQL database. Administrators manage the system via a web-based admin panel built with PHP.

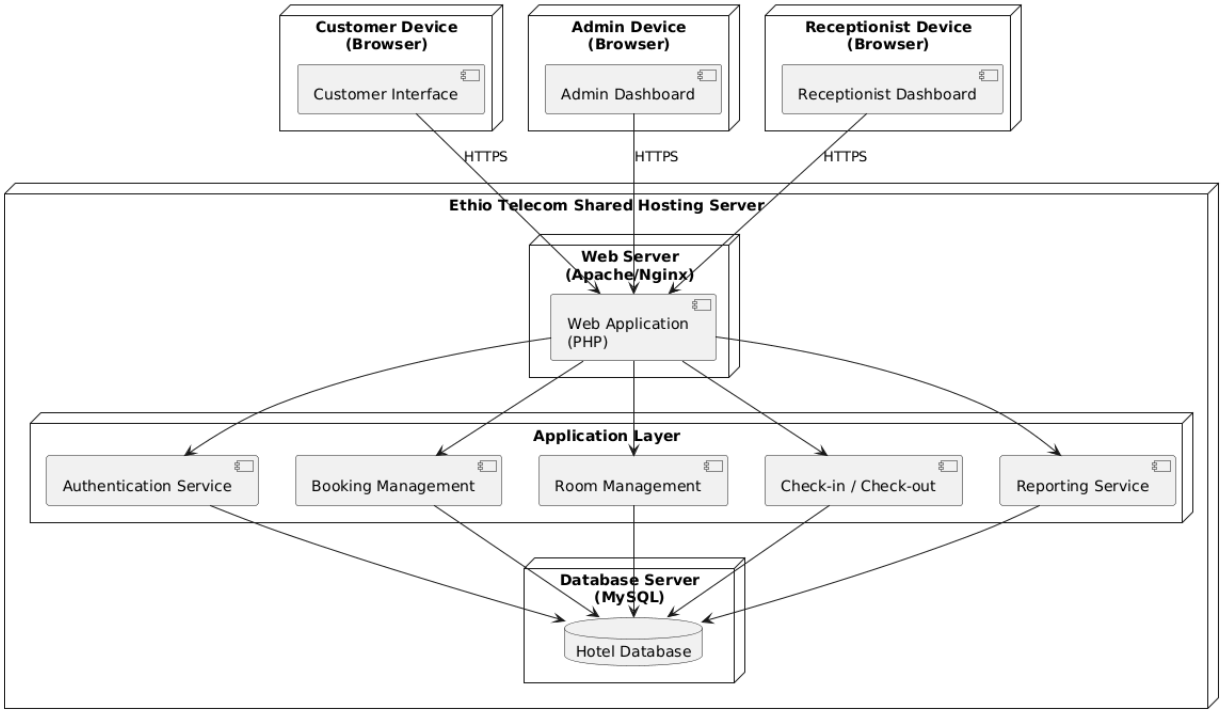


Figure 15 : Hardware/Software Mapping

4.5 State chart Diagram

A state chart diagram is used to describe the dynamic behavior of an object in response to external events throughout its life cycle. It illustrates how an object **changes its state** when specific events occur and how it reacts differently depending on its current state.

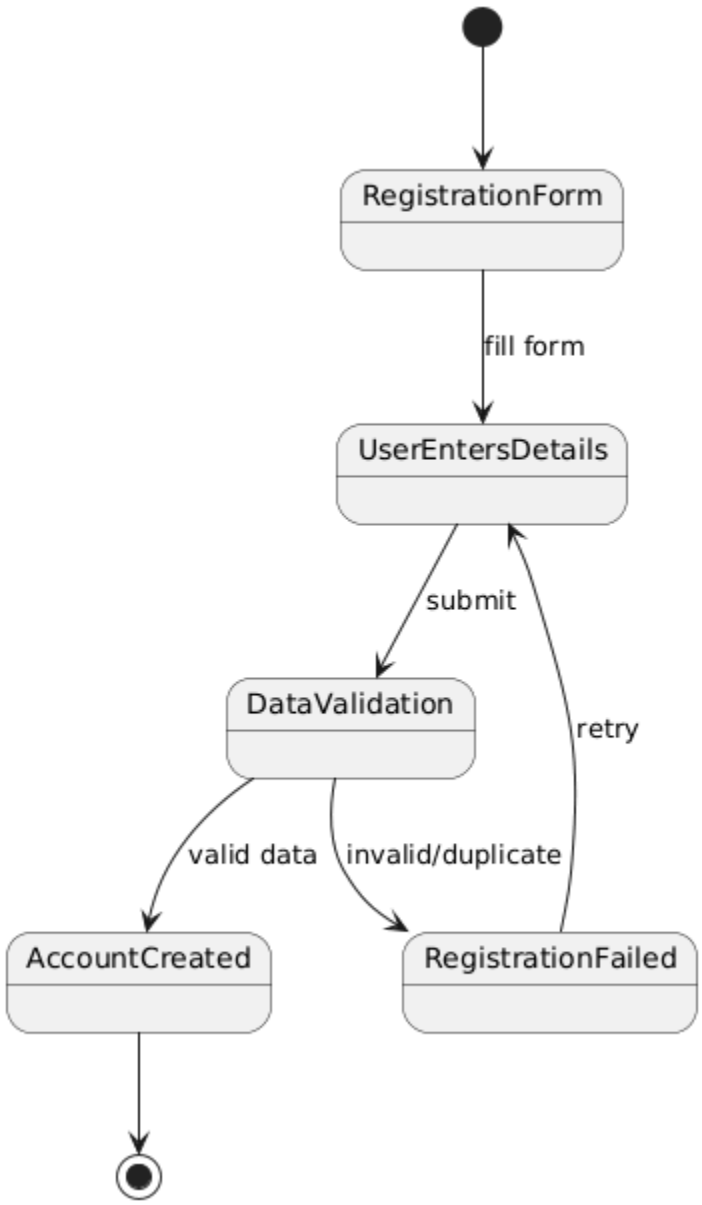


Figure 16 : State chart Diagram for Registration

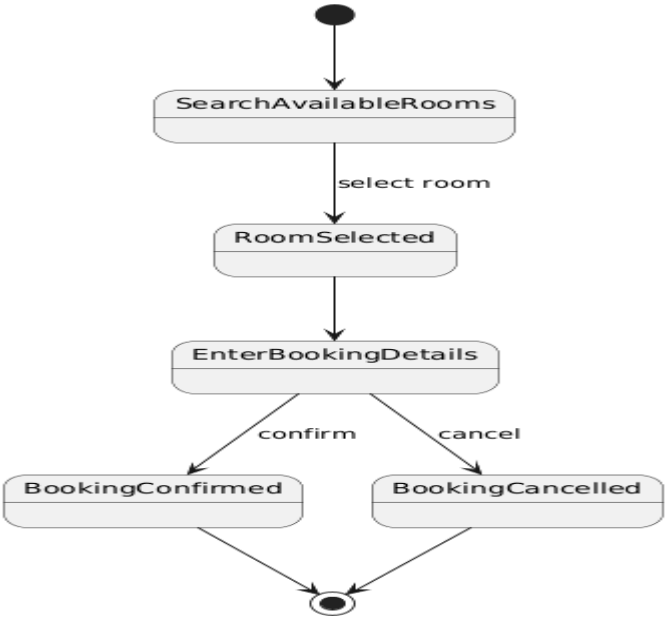


Figure 17 : State chart Diagram for **Room Booking**

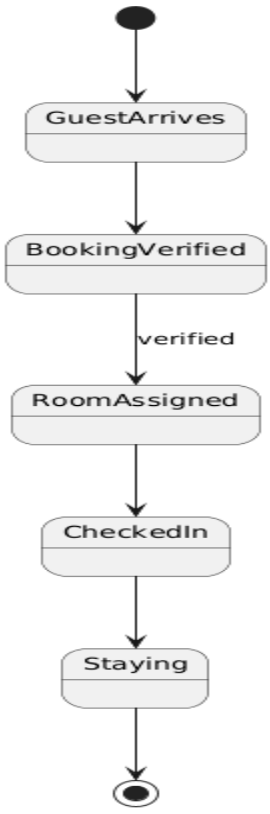


Figure 18 : State chart Diagram for Check-in

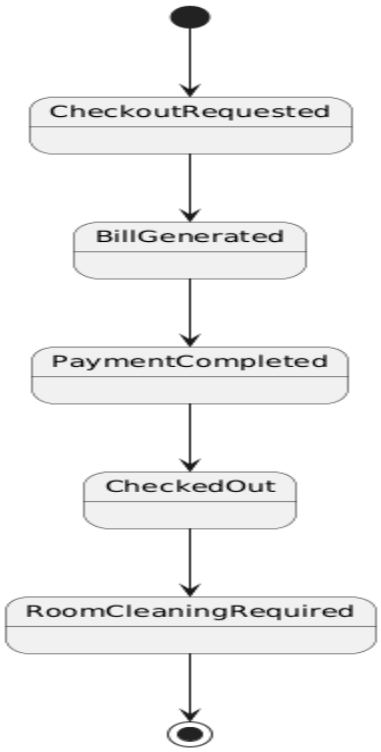


Figure 19 : State chart Diagram for check-out

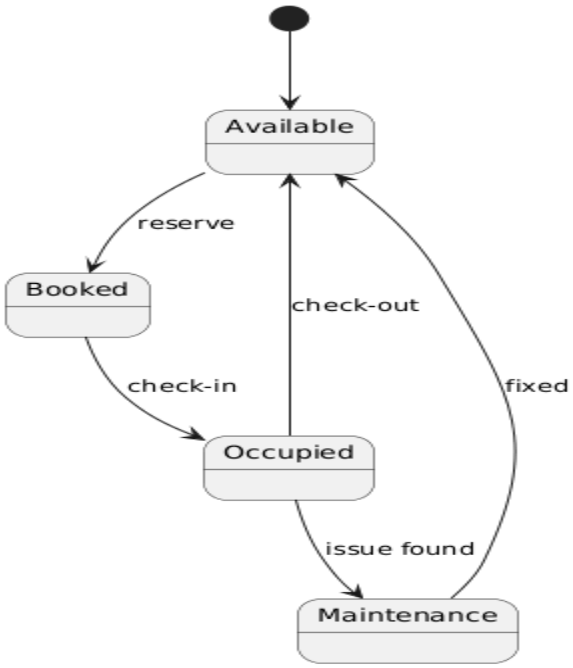


Figure 19 : State chart Diagram Room status

4.6 Collaboration Diagram

A collaboration diagram, also known as a communication diagram, illustrates how multiple objects interact with each other by exchanging messages to complete a specific functionality. These diagrams focus on the structural organization of objects and the sequence of messages exchanged among them.

Unlike sequence diagrams, which emphasize **time order**, collaboration diagrams emphasize **object roles, links, and communication paths**. Messages are numbered to indicate the order in which interactions occur.

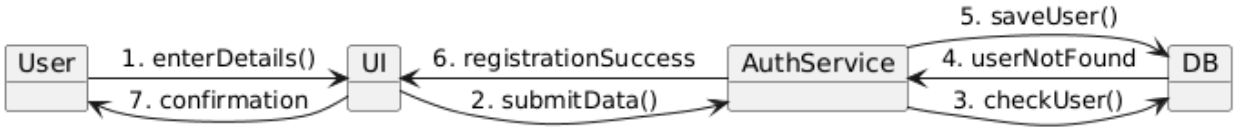


Figure 20 : Collaboration Diagram for Registration

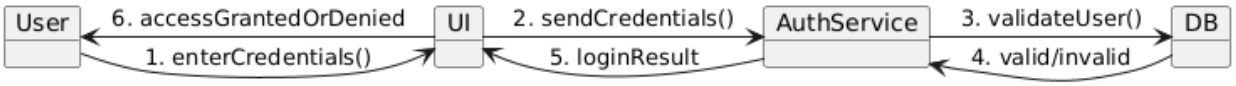


Figure 21 : Collaboration Diagram for Login

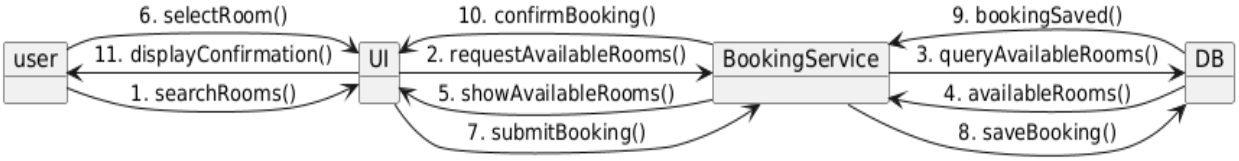


Figure 22 : Collaboration Diagram for Room booking

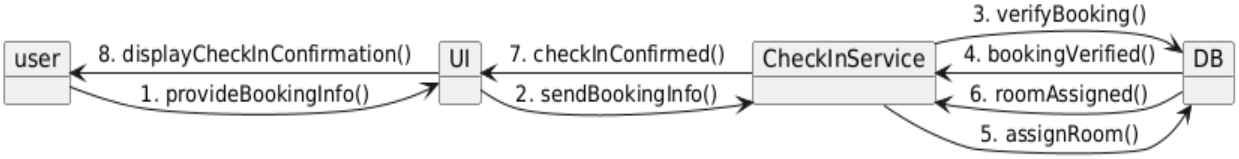


Figure 23 : Collaboration Diagram for chick-in

4.7 Deployment Diagram

A deployment diagram models the physical architecture of a software system. It describes how software components (e.g., web applications, databases) are physically deployed across hardware nodes such as servers, client devices, and networking infrastructure. This helps to understand theruntime environment of the system and how various parts communicate in a real-world deployment.

For Hotel booking system designed for **Teka Nikola Hotel,** a three-tier architecture is used. The deployment involves a **client device**, a **web server** and a **Database server.** Each performing distinct roles. This separation of concerns enhances the system's **scalability**, **maintainability** and **security**.

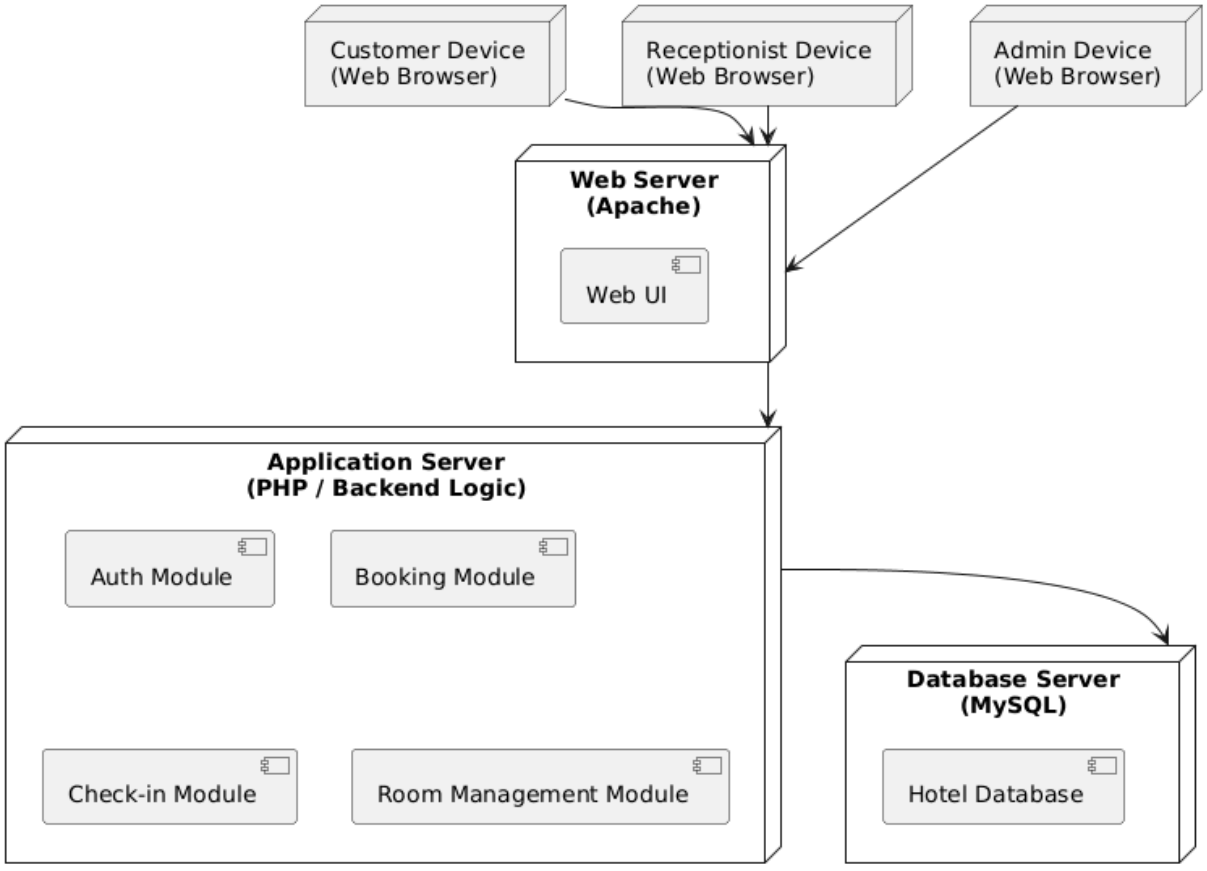


Figure 23 : Deployment Diagram

4.8 Persistent Data Storage and Management

Persistent data storage and management refers to the mechanisms used by the system to **store, retrieve, update, and protect data permanently**, even after the system is shut down or restarted. In the hotel booking system, persistent storage ensures that critical information such as **user accounts, room details, bookings, check-in/check-out records** is securely maintained over time.

The system uses a **relational database management system (RDBMS)**, such as **MySQL**, to manage persistent data. Data is organized into structured tables with defined relationships, enabling **data integrity, consistency, and efficient querying**. Primary keys and foreign keys are used to uniquely identify records and maintain relationships between entities such as users, rooms, and bookings.

Access to persistent data is controlled through the **application layer**, ensuring that users interact with the database only through authorized system functions. This approach improves **security**, **scalability**, and **maintainability** of the system.

Attribute	Data Type	Null	Key	Description
ID	INT AUTO_INCREMENT	NOT NULL	PK	User ID
Fname	VARCHAR(100)	NOT NULL	-	First name
Lname	VARCHAR(100)	NOT NULL	-	Last name
Password	VARCHAR(100)	NOT NULL	-	Encrypted password
Role	ENUM('Customer','Admin','Receptionist')	NOT NULL	-	User role
Phone	VARCHAR(10)	NULL	-	Contact number
Status	ENUM('Active','Suspended')	NOT NULL	-	Account status
ImageURl	VARCHAR(255)	NULL	-	Profile image

Table 35 : Persistent Data Storage for User

Attribute	Data Type	Null	Key	Description
Customer_id	INT AUTO_INCREMENT	PK	NOT NULL	Customer_ID
User_id	INT	FK – User(ID)	NOT NULL	Linked user
Address	VARCHAR(255)	-	NULL	Address
Id_number	VARCHAR(50)	-	NOT NULL	National ID

Table 36 : Persistent Data Storage for Customers

Attribute	Data Type	Null	Key	Description
Admin_id	INT AUTO_INCREMENT	PK	NOT NULL	Admin ID
User_id	INT	FK – User(ID)	NOT NULL	Linked user
Access_level	ENUM('Full','Limited')	-	NOT NULL	Permission level

Table 37 : Persistent Data Storage for Admin

Attribute	Data Type	Null	Key	Description
receptionist_id	INT AUTO_INCREMENT	NOT NULL	PK	Receptionist ID
User_id	INT	NOT NULL	FK – User(ID)	Linked user
Shift	ENUM('Morning','Evening','Night')	NOT NULL	-	Work shift

Table 38 : Persistent Data Storage for Receptionists

Attribute	Data Type	Null	Key	Description
Room_id	INT AUTO_INCREMENT	NOT NULL	PK	Room ID
Room_number	VARCHAR(10)	NOT NULL	UNIQUE	Room number
Room_type	ENUM('Single','Double','Suite')	NOT NULL	-	Room type
Price	DECIMAL(10,2)	NOT NULL	-	Price per night
Status	ENUM('Available','Booked','Occupied','Maintenance')	NOT NULL	-	Room status

Table 39 : Persistent Data Storage for Rooms

Attribute	Data Type	Null	Key	Description
booking_id	INT AUTO_INCREMENT	NOT NULL	PK	Booking ID
customer_id	INT	NOT NULL	FK - Customer(Customer_id)	Booking customer
room_id	INT	NOT NULL	FK – Rooms(Room_id)	Booked room
check_in_date	DATE	NOT NULL	-	Planned check-in
check_out_date	DATE	NOT NULL	-	Planned check-out
booking_status	ENUM('Confirmed','Cancelled')	NOT NULL	-	Booking status
booking_date	DATETIME	NOT NULL	-	Booking time

Table 40 : Persistent Data Storage for Bookings

Attribute	Data Type	Null	Key	Description
checkin_id	INT AUTO_INCREMENT	NOT NULL	PK	Check-in ID
booking_id	INT	NOT NULL	FK – Bookings (booking_id)	Related booking
receptionist_id	INT	NOT NULL	FK – Receptionists (receptionist_id)	Handled by
checkin_time	DATETIME	NOT NULL	-	Actual check-in
Status	ENUM('CheckedIn')	NOT NULL	-	Check-in status

Table 41 : Persistent Data Storage for **Check_In**

Attribute	Data Type	Null	Key	Description
checkout_id	INT AUTO_INCREMENT	NOT NULL	PK	Check-out ID
checkin_id	INT	NOT NULL	FK – **Check_In(**checkin_id**)**	Related check-in
checkout_time	DATETIME	NOT NULL	-	Actual check-out
total_amount	DECIMAL(10,2)	NOT NULL	-	Final bill
Status	ENUM('CheckedOut')	NOT NULL	-	Check-out status

Table 42 : Persistent Data Storage for Check_Out

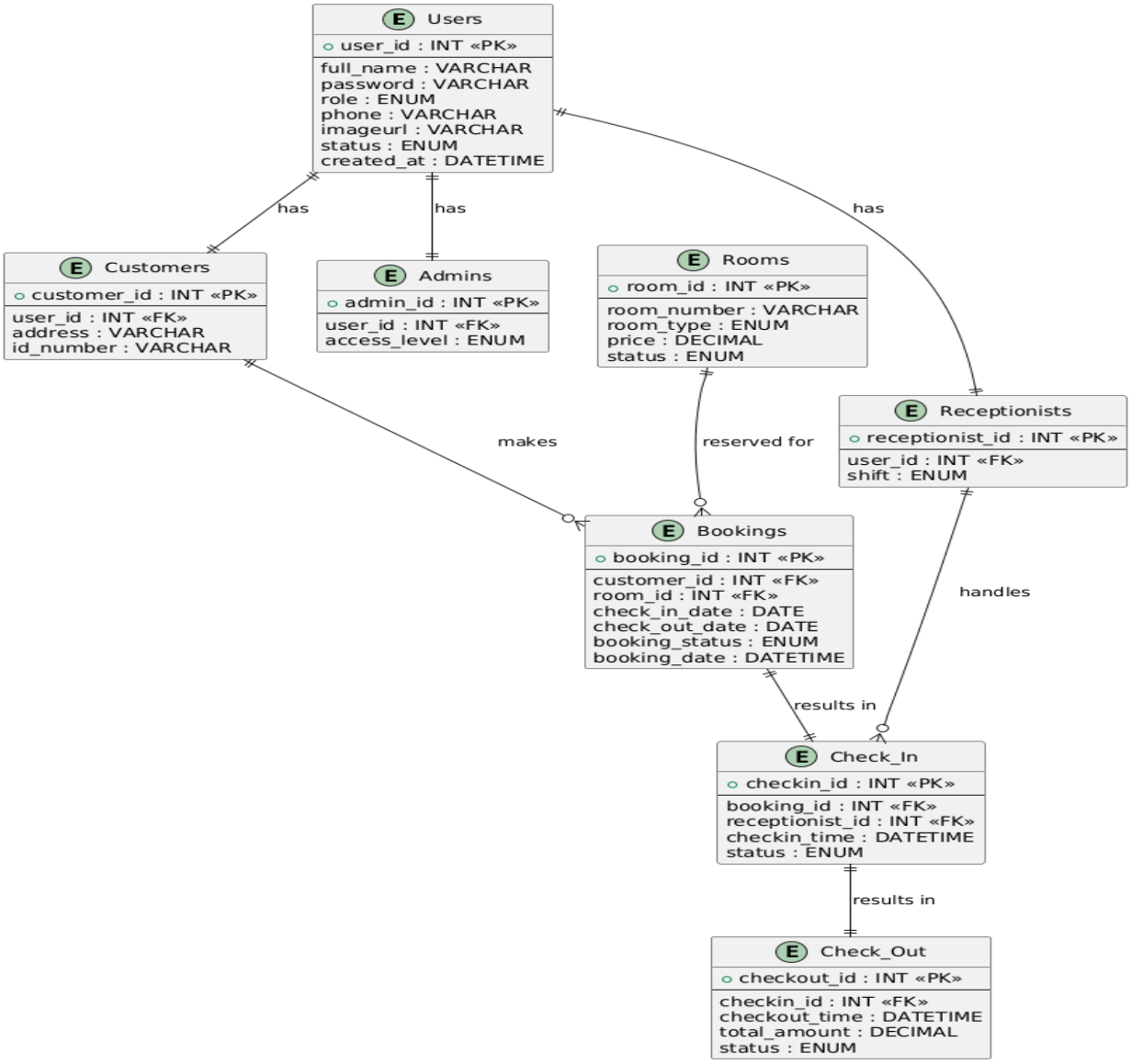


Figure 24 : ER Diagram

4.9 Graphical User Interface Design

The Graphical User Interface (GUI) Design is a critical aspect of the electronics online shopping platform, as it defines how users interact with the system.

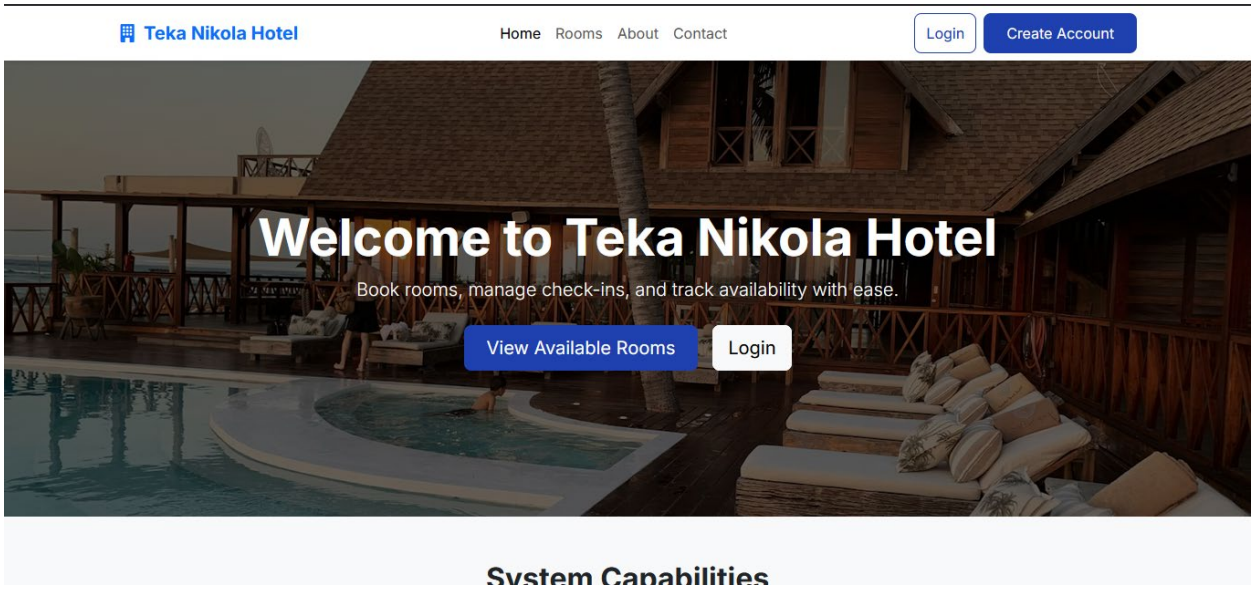


Figure 25 : Graphical User Interface Design

CHAPTER FIVE: CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

The **Teka Nikola Hotel Booking System** is designed to address the limitations of the existing manual and semi-automated hotel operations by providing a centralized, web-based solution. The proposed system improves efficiency in room booking, check-in and check-out processes, user management, and room status tracking.

By supporting three distinct user roles—**Customer, Receptionist, and Admin**—the system ensures proper role-based access and responsibility separation. Customers can easily view room availability and make bookings, receptionists can efficiently manage guest check-ins and check-outs, and administrators can maintain system data such as rooms and users.

The adoption of a structured development methodology and well-defined system architecture enhances reliability, security, and maintainability. Additionally, the exclusion of payment functionality keeps the system focused on core hotel operations, making it suitable for both academic implementation and real-world adaptation.

Overall, the proposed system provides a scalable, user-friendly, and efficient platform that modernizes hotel management at **Teka Nikola Hotel**, reduces human errors, improves service delivery, and lays a strong foundation for future enhancements such as online payments and reporting features.

5.2 Recommendations

1. **System Deployment and Training**
   It is recommended that the system be deployed on a reliable hosting service with adequate security and backup mechanisms. Hotel staff, especially receptionists and administrators, should receive proper training to effectively use the system and minimize operational errors.
2. **Security Enhancement**
   Strong authentication mechanisms, encrypted password storage, and role-based access control should be strictly enforced to protect user data and prevent unauthorized access.
3. **Regular Data Backup** The hotel should implement regular database backup procedures to prevent data loss due to system failure, accidental deletion, or cyber threats.
4. **System Maintenance and Updates** Continuous system maintenance is recommended to fix bugs, improve performance, and adapt to changing hotel requirements. Periodic updates will help keep the system stable and secure.
5. **Future Feature Expansion** Although payment functionality is excluded in the current version, future versions of the system may integrate online payment gateways, reporting dashboards, and customer feedback modules to enhance overall service quality.
6. **Scalability Consideration** The system should be designed in a way that allows easy expansion if the hotel increases the number of rooms or adds additional branches in the future.